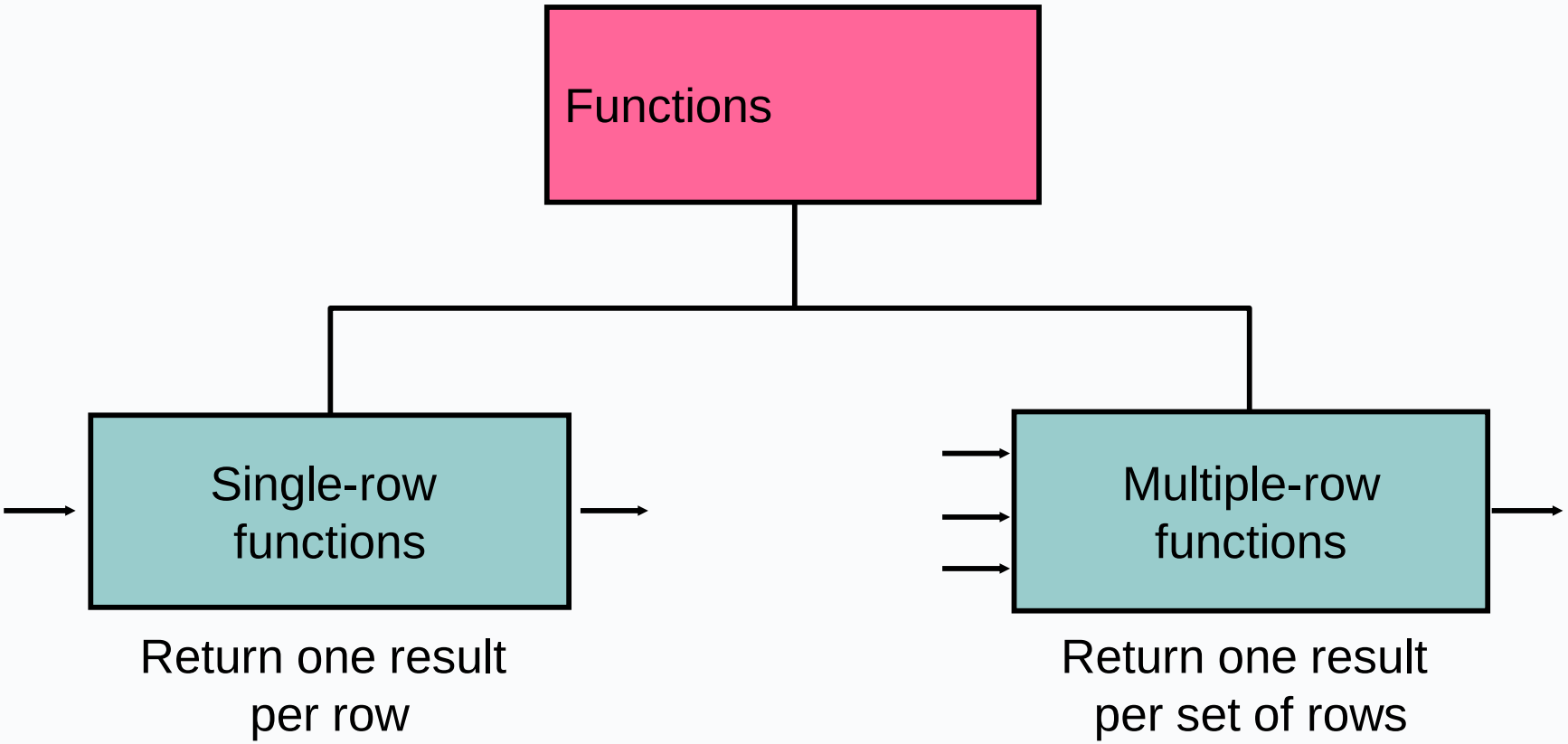
# SQL Functions – Aggregate functions

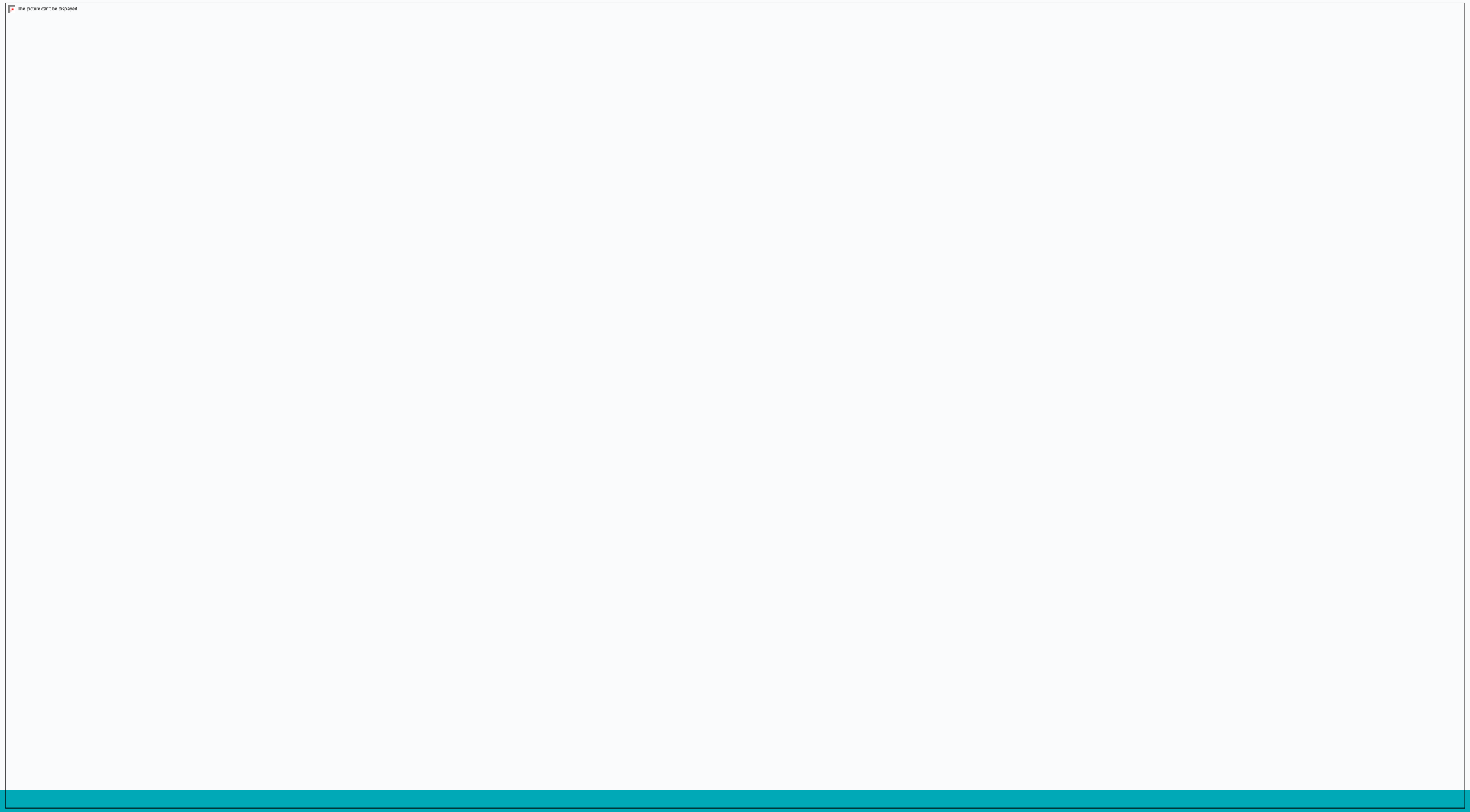
**Dr Mariana Rocha**
**School of Computer Science**

CPMU 2007 Databases 1, Lecture 6

# Types of SQL Functions



# EDA - Football



# Remember: SQL has single and multiple row functions

| Type | Description | Example |
|---|---|---|
| **Single-row** | Act on individual rows and return one value per row. | ROUND(wage), LENGTH(player_name), UPPER(club_name) |
| **Multiple-row** | Combine multiple rows and return a single summary value. | AVG(wage), SUM(wage), COUNT(*) |

Aggregates collapse many rows into one. If you mix aggregated and non-aggregated columns, you must use a GROUP BY clause (we will see an example).

# The count( ) function

- The count( ) function counts how many rows are available in a table.
- The * symbol means all, so we are counting all rows without setting up a specific condition. That results in a single row with the number of table rows.

```
SELECT COUNT(*) FROM table_name;
```

# Group functions and NULL values

- In general, NULL functions will **ignore** NULL values by default.
- However, when we use **count(*)**, NULL values are included as we want to count all rows in a table whether they have a value or not.

```
SELECT COUNT(*) "Total players" FROM players;
SELECT COUNT(*) "Total clubs" FROM clubs;
SELECT COUNT(*) "Total contracts" FROM player_contracts;
SELECT COUNT(*) "Total attributes" FROM player_attributes;
```

# Cross-table coverage

- The command below uses subqueries to count the number of rows in each table for comparison.

```
SELECT
(SELECT COUNT(player_id) FROM players) "Total number of
players",
(SELECT COUNT(club_id) FROM clubs) "Number of clubs",
(SELECT COUNT(DISTINCT player_id) FROM player_attributes)
"Number of players with attributes",
(SELECT COUNT(DISTINCT player_id) FROM player_contracts) "Number
of players with contracts";
```

# Cross-table coverage

- Using distinct, we can gather unique values for a specific attribute.

```
-- Count only contracts that have a non-null wage
SELECT COUNT(wage) "Contracts with wage" FROM player_contracts;


-- How many distinct clubs appear in the contracts table?
SELECT COUNT(DISTINCT club_id) "Unique wage values" FROM
player_contracts;
```

# Creating Groups of Data: GROUP BY

- When our select query mixes single row values with aggregate results, we must use GROUP BY, a statement used to aggregate specific values of an attribute according to a certain group.
- For example: if I want to count how many players each club has, I need to group the count values according to the club_name.

```
SELECT
club_name,
COUNT(player_contracts.player_id) "Players with contract"
FROM clubs
LEFT JOIN player_contracts USING (club_id)
GROUP BY club_name;
```

# Grouping by multiple groups

- You can group by more than one attribute to get summaries at different levels.
- In this example, we are calculating the average wage of players based on their club name and nationality. You will have duplicates on the nationality, as players from the same nationality can be in different clubs.
- We then order the data by two attributes: first, by nationality, then by average wage.

```
SELECT
club_name,
nationality,
ROUND(AVG(wage), 2) "Average wage"
FROM player_contracts
JOIN players USING (player_id)
JOIN clubs USING (club_id)
GROUP BY club_name, nationality
ORDER BY 2, 3 DESC;
```

# The avg( ) function

- The average functions will present numeric values. Suppose we want to know the average wage for players according to the club they play for:

```
SELECT
club_name,
ROUND(AVG(player_contracts.wage), 2) "Average wage"
FROM clubs
JOIN player_contracts USING (club_id)
GROUP BY club_name
ORDER BY 2 DESC;
```

# The avg( ) function

- I can also check the average for the overall rating according to the nationality:

```
SELECT
nationality,
AVG(overall) "Average overall"
FROM players
JOIN player_attributes USING (player_id)
GROUP BY nationality
ORDER BY 2 DESC;
```

# Combining avg( ) and count( )

- We can also combine functions for more complex queries.
- Suppose I want to get the average age of each player considering their nationality and how many players I have for each nationality:

```
SELECT nationality "Nationality",
COUNT(*) "Number of players",
ROUND(AVG(age), 1) "Average age"
FROM players
GROUP BY nationality
ORDER BY 2 DESC;
```

# The sum( ) function

- The sum function will present numeric values.
- Suppose you want to know how much each club spends on their players' wages:

```
SELECT
clubs.club_name,
SUM(wage) "Wage bill"
FROM clubs
JOIN player_contracts USING (club_id)
GROUP BY club_name
ORDER BY 2 DESC;
```

# min( ) and max( )

- These functions will give you the extreme values for an attribute.
- For example, we can retrieve the minimum and maximum wage a player can get.

```
SELECT
MIN(wage) "Minimum wage",
MAX(wage) "Maximum wage"
FROM player_contracts;
```

# min( ) and max( ) with dates

- We can use min( ) and max( ) with date data type arguments.

```
SELECT
MIN(joined) "Earliest contract",
MAX(contract_valid_until) "Latest contract"
FROM player_contracts;
```

# min( ) and max( ) with string

- When dealing with text, min and max consider the order of the alphabet.

```
SELECT
MIN(player_name),
MAX(player_name)
FROM players;
```

# HAVING

- So far, we have been using the command WHERE when applying a specific condition for data retrieval.
- However, that will not work when this condition includes an aggregate function. Let's try the example below:

```
-- Clubs with average wage higher than 50,000
SELECT club_id, ROUND(AVG(wage), 2) "Average wage"
FROM player_contracts
GROUP BY club_id
WHERE AVG(wage) > 50000;
```

# WHERE and HAVING

- An SQL query is executed in this order:

**FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY**

- WHERE applies the specific condition BEFORE the data is grouped. In the previous example, it tried to filter the avg(wage) before the values were grouped according to the club_id.
- HAVING selects group rows after groups and aggregates are computed.
- The WHERE clause must not contain aggregate functions, as it makes no sense to use an aggregate to determine which rows will be inputs to the aggregates.

# Restricting group results with HAVING

- When you use the HAVING clause, the PostgreSQL server restricts groups as follows:
  1. Rows are grouped by the expression you have given.
  2. The group function is applied.
  3. Groups matching the HAVING clause are displayed.

# FILTER + WHERE

- The FILTER clause in SQL is used with aggregate functions to apply conditions to specific rows **before performing the aggregation**.

- The FILTER (WHERE …) clause limits which rows are included in a specific aggregate calculation,
without affecting the rest of the query.

# Filter

- In the following example, we will retrieve:
  - The number of players
  - The number of players with wage
  - The number of players without the wage

```
SELECT
club_name FILTER (WHERE club_name IS NOT NULL),
COUNT(player_id),
COUNT(player_id) FILTER (WHERE wage IS NOT NULL) "With wage",
COUNT(player_id) FILTER (WHERE wage IS NULL) "Without wage"
FROM clubs
LEFT JOIN player_contracts USING (club_id)
GROUP BY club_name
ORDER BY club_name;
```

# FILTER x HAVING

- FILTER and HAVING might sound similar, but they are not interchangeable.
- We use FILTER when we want multiple aggregates with different conditions in the same result row.
- Use HAVING when you want to filter out entire groups based on aggregate results.

# Nesting group functions – not allowed

- SQL does not allow an aggregate function to be nested inside another aggregate function in the same query block.
- This happens because aggregates operate on sets of rows, not on single values.
- SQL needs to know when the aggregation ends before another begins — but nested aggregates blur that boundary.

# Nesting group functions – not allowed

- The following returns an error:

```
SELECT
MAX(AVG(wage))
FROM player_contracts;
```

- AVG(wage) → wants to compute an average across all rows.
- MAX(...) → also wants to scan across all rows, but now the input would be the averages of groups that don’t exist yet.
- The SQL engine doesn’t have a stage in its pipeline where both can coexist without ambiguity.
- Aggregation happens once per query block, right after GROUP BY is evaluated.

# Nesting group functions – not allowed

- We need to use a subquery to retrieve that data:

```
--We can retrieve that data using a subquery
SELECT ROUND(MAX("Average wage"), 2) "Maximum wage" --notice I
can nest round and max because round is a single-row function
FROM (
SELECT AVG(wage) "Average wage"
FROM player_contracts
GROUP BY club_id
) "Subquery";
```

# String aggregation

- String aggregation works similarly to the concatenate function, but allows aggregation of multiple rows using the GROUP BY.

- Let’s aggregate all player full names in a single comma-separated string per club, alphabetically.

```
SELECT
club_name,
STRING_AGG(player_name, ', ' ORDER BY players.player_name)
"Players list"
FROM clubs
JOIN player_contracts USING (club_id)
JOIN players USING (player_id)
GROUP BY club_name
ORDER BY club_name;
```

# Dashboard for overview

```
SELECT
club_name,
COUNT(DISTINCT player_id) "Number of players",
ROUND(AVG(wage), 2) "Average wage",
SUM(wage) "Total wage bill",
MIN(wage) "Minimum wage",
MAX(wage) "Maximum wage",
COUNT(*) FILTER (WHERE contract_valid_until <=
EXTRACT(YEAR FROM CURRENT_DATE)) "Number of valid
contracts",
STRING_AGG(DISTINCT player_name, ', ' ORDER BY
player_name) "Players list"
FROM clubs
LEFT JOIN player_contracts USING (club_id)
LEFT JOIN players USING (player_id)
GROUP BY club_name
ORDER BY 3 DESC NULLS LAST;
```


vevox
What do you think makes a good team?
good culture
structure
encouragement
hear other voices
openness
inclusivity
be proactive
values
supportive
respect
clarity
safety
honesty
trust
foster collaboration
fairness
good communication
kindness
Join at:
vevox.app
ID:
123-456-789
Poll Open
What do you think makes a good team?
honesty